

# Projet S8 : Banc de test avionique

Tom Chassing

Semestre 8 — 29 Mai 2026

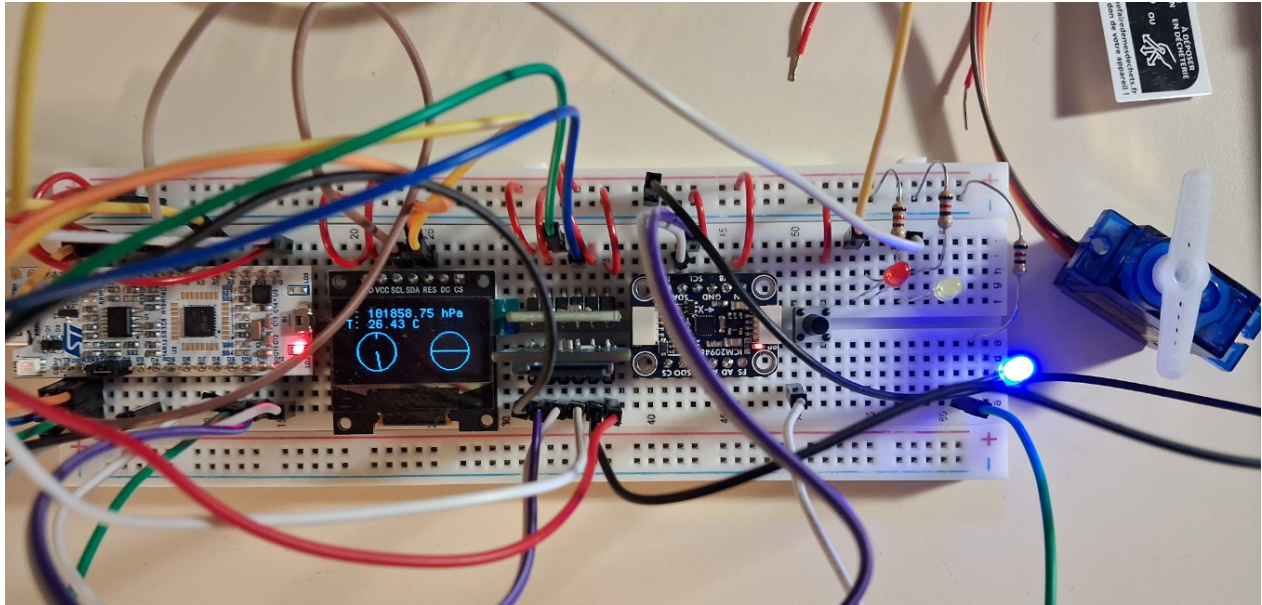


FIGURE 1 – vue du banc de test avionique, en état de pause sur l’acquisition

## Introduction

Dans le cadre de la matière Systèmes microprogrammés (4TIM812U) du M1 du CMI IMSAT parcours systèmes embarqués, nous avons chacun un projet de banc de test avionique à construire.

Pour ce faire, telles étaient nos instructions :

- FP1 Afficher sur l’écran OLED et sur le terminal série la pression en hPa, la température en degrés Celsius et les angles de roulis, tangage et lacet en degrés avec une fréquence d’acquisition de 5 Hz.
- FP2 Enregistrer les données sur la carte microSD.
- FP3 Générer la couche physique d’une communication au format ARINC429 avec la donnée de pression. Le label choisi sera 256, les bits concernant les champs SDI et SSM seront mis à 0.
- FP4 Indiquer le lacet avec le servomoteur.
- FP5 Indiquer les dépassements de limites de roulis et de tangage avec des LEDs. Les angles doivent rester entre  $-40^\circ$  et  $+40^\circ$ .

# Table des matières

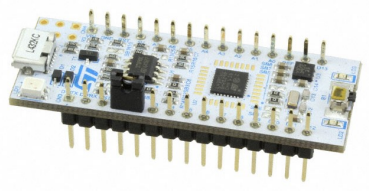
|           |                                                                              |           |
|-----------|------------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Matériel</b>                                                              | <b>3</b>  |
| 1.1       | Liste du matériel . . . . .                                                  | 3         |
| 1.2       | Détails des broches de la STM32L432KC . . . . .                              | 4         |
| <b>2</b>  | <b>GitHub et chronologie</b>                                                 | <b>5</b>  |
| 2.1       | Chronologie du développement . . . . .                                       | 5         |
| <b>3</b>  | <b>Algorithme</b>                                                            | <b>6</b>  |
| <b>4</b>  | <b>I2C</b>                                                                   | <b>7</b>  |
| 4.1       | Information sur l’I2C . . . . .                                              | 7         |
| 4.2       | Exemple du BMP280 . . . . .                                                  | 7         |
| 4.3       | Autres composants communicants en I2C : . . . . .                            | 9         |
| <b>5</b>  | <b>SPI</b>                                                                   | <b>10</b> |
| 5.1       | Information sur le SPI . . . . .                                             | 10        |
| 5.2       | Exemple du Pmod MicroSD . . . . .                                            | 10        |
| <b>6</b>  | <b>UART</b>                                                                  | <b>14</b> |
| 6.1       | Information sur l’UART . . . . .                                             | 14        |
| 6.2       | Mise en place de l’UART . . . . .                                            | 14        |
| 6.3       | Au niveau du code . . . . .                                                  | 15        |
| <b>7</b>  | <b>PWM : Modulation de Largeur d’Impulsion</b>                               | <b>16</b> |
| 7.1       | Informations sur le PWM . . . . .                                            | 16        |
| 7.2       | Mise en place . . . . .                                                      | 16        |
| 7.3       | L’implémentation dans le code . . . . .                                      | 17        |
| <b>8</b>  | <b>ARINC</b>                                                                 | <b>18</b> |
| 8.1       | Présentation de l’ARINC429 . . . . .                                         | 18        |
| 8.2       | Implémentation dans le code . . . . .                                        | 18        |
| <b>9</b>  | <b>Mise en place d’une interruption</b>                                      | <b>20</b> |
| 9.1       | Création d’un flag . . . . .                                                 | 20        |
| 9.2       | Limitation du nombre d’acquisition . . . . .                                 | 20        |
| 9.3       | Ajout d’une LED d’état d’acquisition — Programmation en bare metal . . . . . | 21        |
| <b>10</b> | <b>Validation et résultats</b>                                               | <b>23</b> |
| <b>11</b> | <b>Bibliographie</b>                                                         | <b>25</b> |
| <b>12</b> | <b>Annexe — Réponses aux questions</b>                                       | <b>26</b> |

# 1 Matériel

## 1.1 Liste du matériel

Avant d'attaquer sur le développement et les résultats, pour mieux comprendre la suite du rapport, je me dois de présenter le matériel utilisé pour le projet :

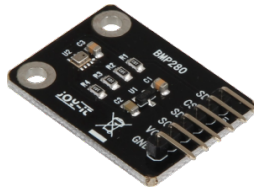
- Micro-contrôleur STM32 NUCLEO-L432KC de STMicroelectronics (*donc utilisation du logiciel STM32CubeIDE*)



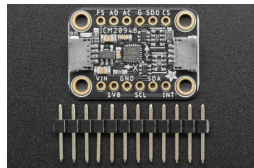
- Ecran OLED SBC-OLED01V2



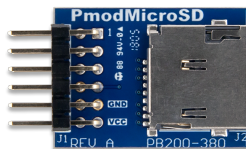
- Capteur de Pression / Température BMP280



- Gyroscope / Accéléromètre / Compas magnétique ICM-20948 9-DoF



- Digilent Pmod MicroSD



- Servomoteur TowerPro SG90



## 1.2 Détails des broches de la STM32L432KC

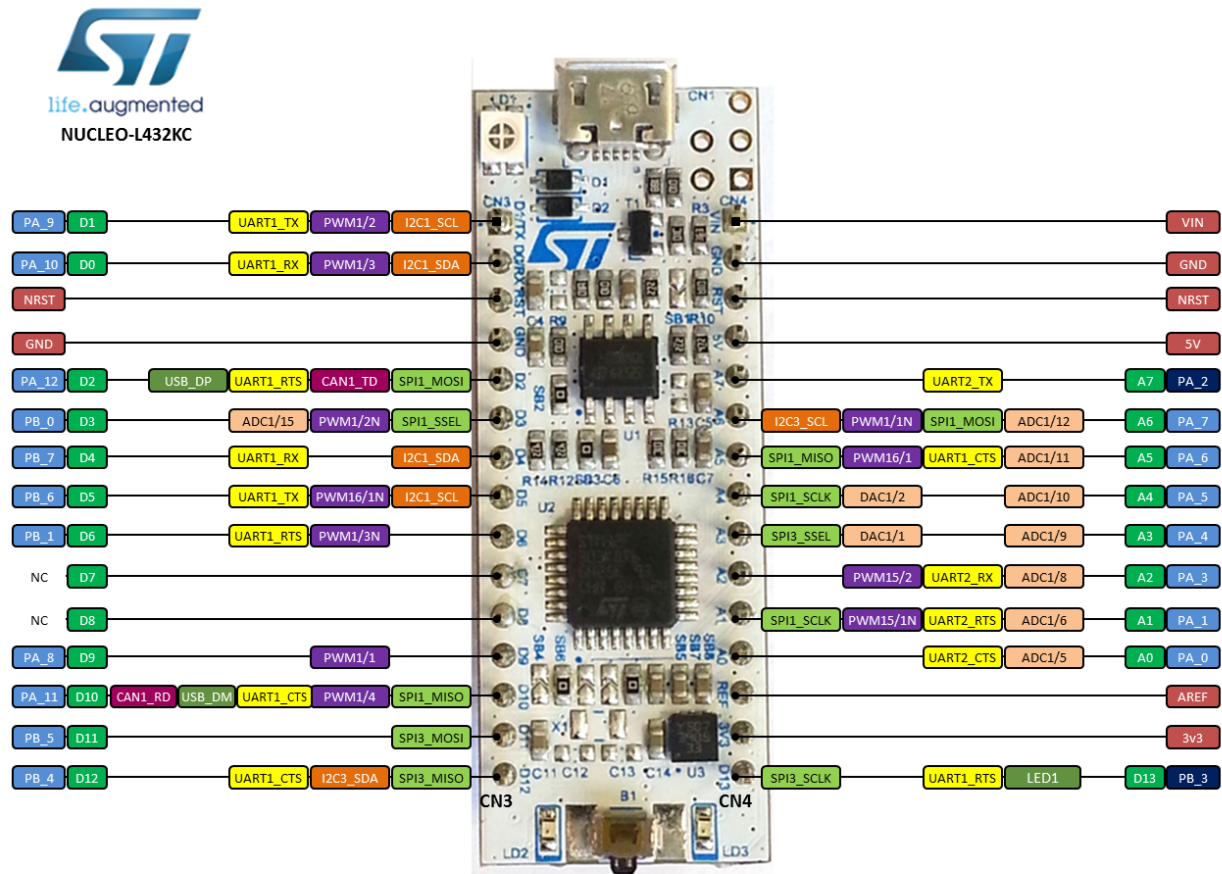


FIGURE 2 – Pinout de la carte STM32L432KC

| Broche (Pin) | Label / Fonction |
|--------------|------------------|
| PA1          | GPIO_pitch       |
| PA2          | VCP.TX           |
| PA3          | GPIO_roll        |
| PA6          | TIM16_CH1        |
| PA8          | BP_GPIO_EXTI8    |
| PA9          | I2C1_SCL         |
| PA10         | I2C1_SDA         |
| PA15         | VCP.RX           |
| PB0          | SPI3.CS          |
| PB3          | SPI3.SCK         |
| PB4          | SPI3.MISO        |
| PB5          | SPI3.MOSI        |

TABLE 1 – Résumé du Pinout du STM32L432KCx

## 2 GitHub et chronologie

Comme demandé pour le projet, nous avons dû ouvrir un dépôt GitHub sur lequel téléverser nos codes au fur et à mesure du développement.

*Lien de mon dépôt GitHub*

### 2.1 Chronologie du développement

Dans mon repère, nous pouvons trouver 7 branches, auxiliaires au main. Elles reflètent l'ordre de développement, malgré l'affichage non-chronologique de GitHub, et qui se sont basées sur les "FP" des instructions en introduction. Chronologiquement :

- **etape1** : implémentation de la lecture des données de pression et de température du capteur BMP280, et affichage de ceux-ci sur l'écran OLED ;
- **etape2** : transmission des données sur une carte microSD ;
- **etapeGyro** : ajout de l'ICM-20948 comprenant un gyroscope, un accéléromètre et un compas magnétique ;
- **etape4** : ajout d'un servomoteur, dont le mouvement est dicté par les données du lacet (cap magnétique) ;
- **etape5** : indication, au travers de diodes électroluminescentes (LEDs), des dépassement de limite comprises entre  $-40^\circ$  et  $+40^\circ$  du tangage et du roulis ;
- **etapeARINC** : ajout d'une communication au format ARINC429 (*norme aéronautique*), contenant la mesure de la pression, sous le label 256 ;
- **Perfectionnement** : perfectionnement de certaines fonctions, et ajout d'une LED donnant l'état de l'acquisition (*acquisition en cours, mise en pause, terminée*).

**Commentaire** On peut noter un suivi non-chronologique des instructions puisque j'ai ajouté la centrale inertielle (**etapeGyro**) après la transmission à la carte microSD, et l'ARINC après l'instruction FP5. Également, j'ai effectué une étape de perfectionnement, reprenant les étapes précédentes pour fournir un travail plus précis (*ex : affichage des données des angles de la centrale inertielle en degrés, plutôt que de les donner en valeurs brutes*), mais cela m'a aussi été utile pour tester la programmation en bare metal (*programmation non-complexe à l'image de l'Assembleur*).

### 3 Algorithme

Voici le diagramme de mon algorithme qui explicite le cycle de vie du système de banc de test.

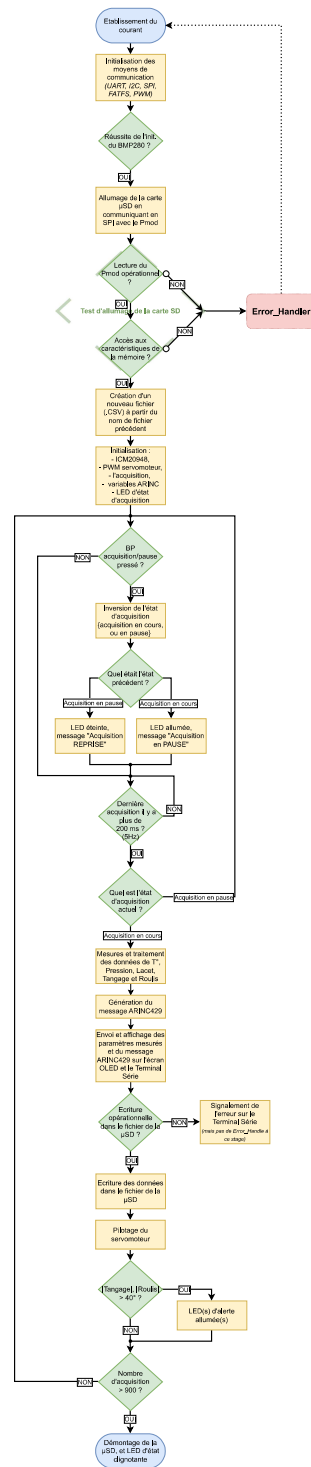


FIGURE 3 – Algorithme de mon banc de test avionique

## 4 I2C

### 4.1 Information sur l'I2C

Pour la lecture de données des capteurs BMP280 et ICM-20948, et l'affichage sur l'écran OLED, j'ai dû utiliser la communication I2C. L'I2C est un bus série synchrone bidirectionnel half-duplex, où plusieurs équipements, maîtres ou esclaves, peuvent être connectés au bus. De par sa caractéristique bidirectionnel half-duplex, il n'y a besoin que de deux fils de communication : le SDA (**S**erial **D**ata **L**ine) qui est la ligne bidirectionnelle de données, et le SCL (**S**erial **C**lock **L**ine) qui est la ligne bidirectionnelle d'horloge de synchronisation.

*Lien de la documentation pour les branchements au micro-contrôleur.*

Dans mon cas, j'ai relié toutes les SDA des composants à la broche D0 (PA-10) et les SCL à la broche D1 (PA-9) ; qui sont toutes deux rattachés à la communication I2C1 de mon ST32L432KC. Contrairement à la communication SPI (que je présenterai juste après l'I2C), je n'ai pas besoin de deux broches par élément esclave car je peux tout relier aux deux mêmes broches communes puisque l'I2C fait usage de l'adressage. De manière un peu plus claire, lorsque j'envoie une commande par l'I2C, je lui fournis l'adresse du composant ciblé et tous les éléments esclave reçoivent cette commande, mais seul le composant dont l'adresse correspond prend en compte la commande.

### 4.2 Exemple du BMP280

Prenons l'exemple du BMP280. Déjà, je précise que j'ai dû utiliser le dépôt GitHub du BME280 qui est quasi-identique au BMP280, mais avec un capteur d'humidité en plus ; j'ai donc juste supprimé les fonctions d'humidité.

Première chose, c'est repérer l'adresse du composant. Généralement, ou de ce que j'ai vu sur mes trois composants en I2C, ils possèdent deux adresses selon si l'on relie la broche SDIO au GND ou au Vcc. Pour le BMP280, c'est 0x76 si le SDIO est relié au GND, ou 0x77 s'il est relié au Vcc comme dans mon cas. Néanmoins, il s'agit de l'adresse 7 bits qui ne prend pas en compte le 8ème bit donnant l'instruction "Write / Read" ('0' pour écrire, '1' pour lire). Ainsi, il nous faut décaler notre adresse et donc nous définissons l'adresse comme  $0x77 \ll 1 = 0xEE$ . Dans notre *BME280\_STM32.h*, j'ai donc dû assigner `hi2c1` au `BME280_I2C` pour utiliser l'I2C1 de mon microcontrôleur, et mettre `0xEE` au `BME280_ADDRESS` pour définir l'adresse du composant.

---

```
#ifndef BME280_STM32_H_
#define BME280_STM32_H_
/*...*/
#define BME280_I2C hi2c1
/*...*/
#define BME280_ADDRESS 0xEE
// Le SDIO est a VCC, donc l'adresse 7 bits est 0x77 et l'adresse 8 bits est 0xEE !
extern I2C_HandleTypeDef BME280_I2C;
```

---

Pour commencer, nous configurons notre micro-contrôleur par l'horloge, les broches ainsi que plusieurs autres paramètres liés à ces broches (ex : résistance de pull-up/-down).

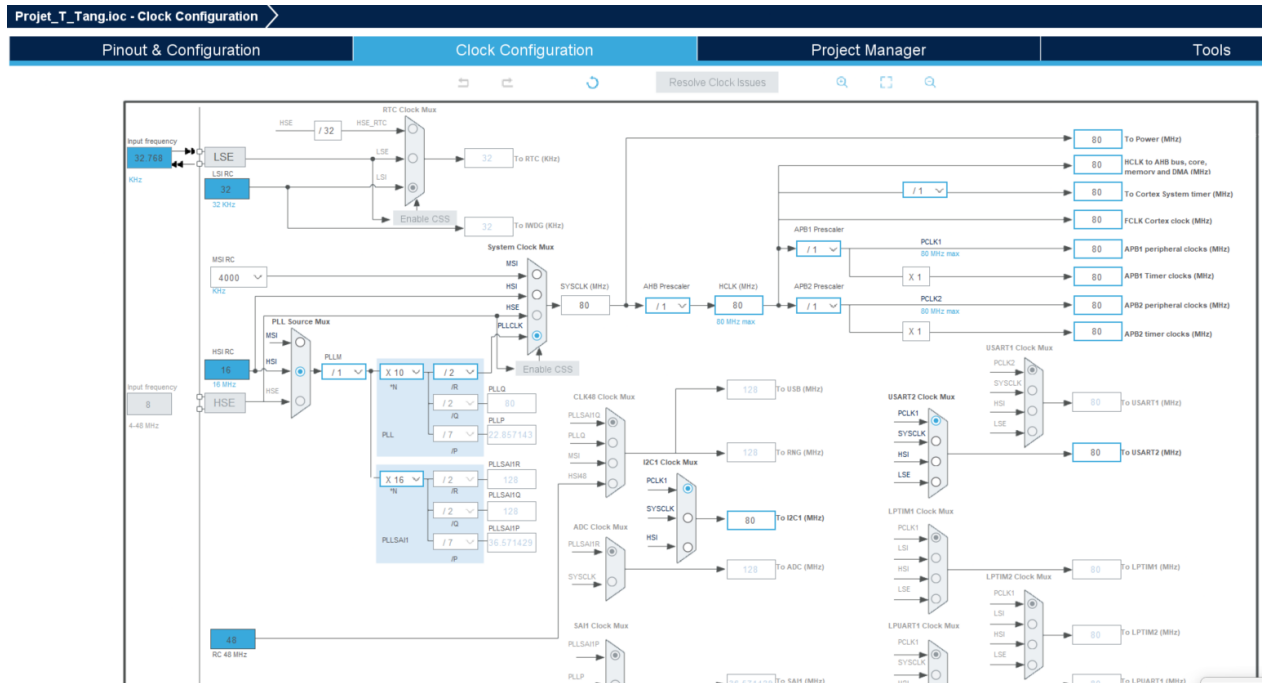


FIGURE 4 – Configuration de l'horloge du STM32L432KC

Sur la figure 3, nous voyons que l'horloge est mise à son max en 80MHz, c'est pour une meilleure précision. En effet, plus l'horloge système est haute et précise (80 MHz), plus le diviseur de l'I2C a du "grain" pour fabriquer un signal propre sur lequel son timing repose.

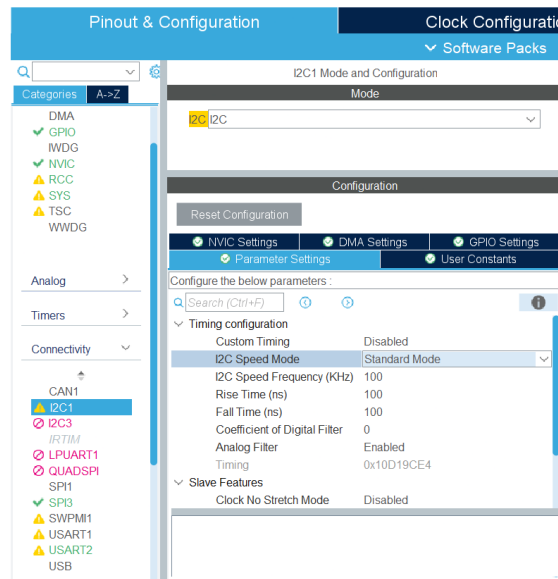


FIGURE 5 – Pinout and Configuration, activation de l'I2C1 dans la rubrique Connectivité



Ensuite après l'activation de l'I2C1, telle la Figure 4, STM32CubeIDE génère les codes dans le main.c et les autres fichiers complémentaires, dont la configuration initiale des broches et connectiques pour nous éviter à devoir le faire manuellement en bare metal. En guise d'exemple, voici ci-dessous le code auto-généré par STM32CubeIDE pour la configuration initiale de notre I2C1 :

---

```

/* * @file           : main.c
 * @brief           : Main program body */
/*...*/
/* USER CODE END I2C1_Init 1 */
hi2c1.Instance = I2C1;
hi2c1.Init.Timing = 0x10D19CE4;
hi2c1.Init.OwnAddress1 = 0;
hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
hi2c1.Init.OwnAddress2 = 0;
hi2c1.Init.OwnAddress2Masks = I2C_OA2_NOMASK;
hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
if (HAL_I2C_Init(&hi2c1) != HAL_OK)
{
    Error_Handler();
}

```

---

Cela nous évite beaucoup de travail. Il nous reste qu'à employer les fonctions du fichier BME280\_STM32.c pour démarrer et configurer comme on le souhaite notre capteur BMP280 :

---

```

int status = BME280_Config (OSRS_2, OSRS_16, MODE_NORMAL, T_SB_0p5, IIR_16);
if (status!= 0)
{
    Error_Handler();
}
BME280_WakeUP();

```

---

Puis de tout simplement lire dans le while(1) du main(void) :

---

```

/* USER CODE BEGIN 3 */
BME280_Measure(&temp,&press);

```

---

### 4.3 Autres composants communicants en I2C :

- l'ICM-20948 : l'adresse est 0xD2 (0x69  $\ll$  1), et j'ai fait usage du dépôt GitHub pour le composant. Cependant, il faut noter que celui-ci est configuré pour une communication SPI, ce que j'ai changé (et ce ne fut pas si dur, car il n'y a à remplacer que les appels HAL\_SPI\_Transmit et \_Receive par les équivalents I2C.

*lien de la documentation pour l'ICM-20948*

- l'écran OLED : l'adresse est 0x78 (0x3C  $\ll$  1), le dépôt GitHub utilisé est celui du SSD1306.

*lien de la documentation utilisée, NB* : La broche RES (Reset), reliée au RST n'est pas une broche liée au protocole I2C lui-même. C'est une commande matérielle directe qui sert à réinitialiser la puce interne de l'écran.

## 5 SPI

### 5.1 Information sur le SPI

Pour l'écriture des données d'acquisition sur la carte microSD, j'ai fait usage de la communication SPI (Serial Peripheral Interface). Contrairement à l'I2C, le SPI est un bus série synchrone bidirectionnel Full-Duplex, ce qui signifie que les données peuvent être émises et reçues simultanément. De plus, il s'agit d'une architecture de type maître-esclave point à ligne ou multipoint, où un seul maître (ici mon STM32L432KC) contrôle un ou plusieurs esclaves.

De par sa caractéristique full-duplex et sa structure, la communication nécessite au minimum quatre fils physiques :

- **MOSI** (Master Out Slave In) : la ligne de transmission de données du maître vers l'esclave ;
- **MISO** (Master In Slave Out) : la ligne de réception de données de l'esclave vers le maître ;
- **SCK** (Serial Clock) : la ligne d'horloge générée exclusivement par le maître pour synchroniser les données ;
- **CS** ou **SS** (Chip Select / Slave Select) : la ligne de sélection matérielle de l'esclave.

Dans mon cas, j'ai configuré la communication SPI3 de mon microcontrôleur. Les lignes communes partagées sont rattachées aux broches D11 (PB-5) pour le MOSI, D12 (PB-4) pour le MISO, et D13 (PB-3) pour le SCK. Contrairement à l'I2C qui utilise un adressage logiciel codé sur les premiers bits de la trame, le protocole SPI utilise une sélection purement matérielle. Pour dialoguer avec la carte SD, le maître doit abaisser manuellement la ligne CS dédiée (reliée à la broche PB-0 dans mon circuit) à l'état bas (0V). Si plusieurs esclaves étaient présents sur ce même bus, les lignes MOSI, MISO et SCK resteraient communes, mais il me faudrait allouer une broche de Chip Select (CS) indépendante pour chaque composant esclave supplémentaire afin de les activer un par un.

### 5.2 Exemple du Pmod MicroSD

Avec la communication SPI, notamment pour utiliser la microSD, il faut faire plus attention à ses paramètres dans le fichier *.ioc*.

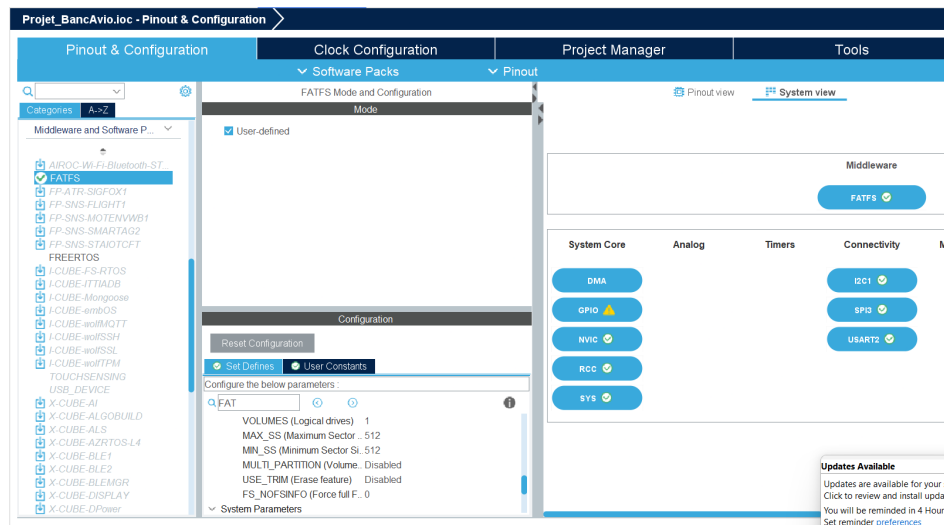


FIGURE 6 – Activation du FATFS

Le FatFS est une couche intermédiaire (un "middleware"). Elle permet au code C de manipuler des fichiers et des dossiers (comme ouvrir, lire et écrire) sur un support physique (notre carte SD ici).

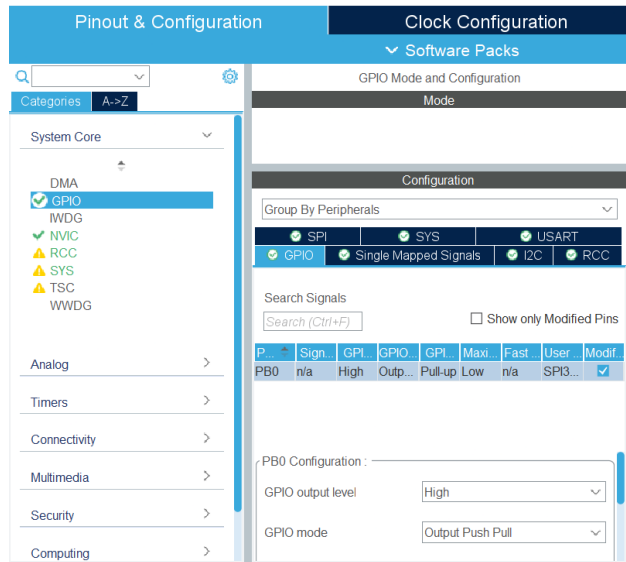


FIGURE 7 – Paramétrage de la broche D3 (PB-0) du Chip Select

Le GPIO output level est mis à 'High' initialement car le CS a un inverseur logique, fonctionnant donc quand le signal du CS est au niveau bas (0v) comme expliqué précédemment. Une résistance de pull-up est également configurée.

Enfin vient la configuration de la connexion SPI3. Plusieurs paramètres à vérifier :

- le 'Full-Duplex Master' comme souhaité ;
- le 'Data Size' mis à 8 bits pour correspondre à notre matériel ;
- le 'Prescaler' mis à 256 (le maximum) pour s'assurer que le 'Baud Rate' soit bien dans l'intervalle [100kHz ; 400kHz] pour que notre Pmod le supporte.

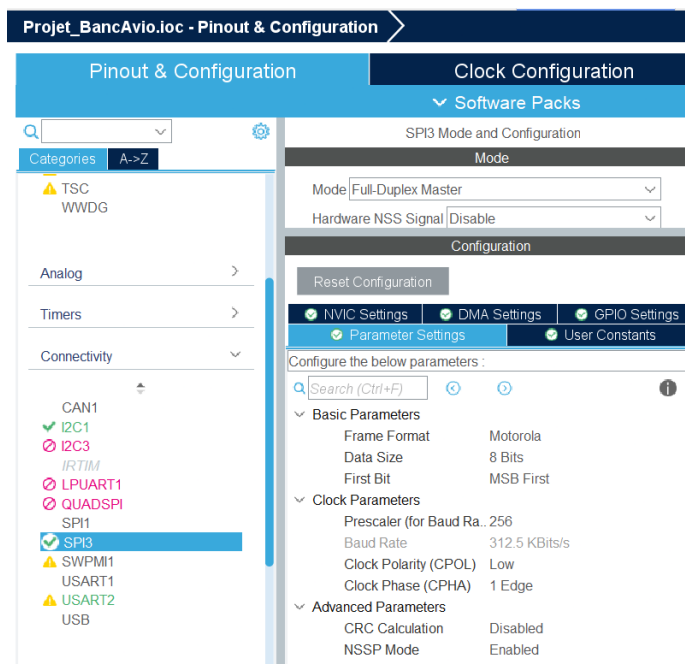


FIGURE 8 – Configuration de la communication SPI

Quant au code, j'ai importé les fichiers *user\_diskio\_spi.c* et *.h* de ce dépôt GitHub du Dr. Hammond Pearce. Pour faire fonctionner les fonctions de ces fichiers, il est important d'associer le port et le pin du Chip Select, tel que :

---

```
//Dans le main.c
#include "fatfs.h"
/*...*/
* USER CODE BEGIN PV */
#define SPI_CS_GPIO_Port GPIOB
#define SPI_CS_Pin GPIO_PIN_0
```

---

Puis il nous faut vérifier la communication entre le Pmod et le micro-contrôleur. Commençons par la pré-initialisation matérielle et réveil de la carte SD :

---

```
//Force CS high
//SD cards require CS to be high when power is applied
HAL_GPIO_WritePin(SPI_CS_GPIO_Port, SPI_CS_Pin, GPIO_PIN_SET);
HAL_Delay(10);
uint8_t tx = 0xFF, rx = 0x00;
myprintf("Dummy bytes received: ");
for(int i = 0; i < 10; i++) {
    HAL_SPI_TransmitReceive(&hspi3, &tx, &rx, 1, 100);
    myprintf("%02X ", rx);
}
```

---

Cette première phase gère l'alimentation et la synchronisation initiale de la carte microSD. À la mise sous tension, les cartes SD nécessitent que leur ligne de sélection matérielle Chip Select (CS) soit maintenue à l'état haut (GPIO\_PIN\_SET) pendant que la tension se stabilise.

De plus, la norme physique des cartes SD impose de fournir un minimum de 74 impulsions d'horloge sur la ligne SCK, avec la ligne de transmission MOSI maintenue à l'état haut ('1'), avant de pouvoir lui envoyer la moindre commande logicielle. Pour remplir cette condition, on lui envoie une salve de 10 octets fictifs (les "Dummy bytes") de valeur 0xFF = 80 impulsions d'horloge. Cette étape force le contrôleur interne de la carte microSD à basculer de son mode de communication natif (SDIO) vers le mode esclave SPI.

Envoyons-lui maintenant une Commande de réinitialisation logicielle (CMD0) :

---

```
HAL_GPIO_WritePin(SPI_CS_GPIO_Port, SPI_CS_Pin, GPIO_PIN_RESET);
HAL_Delay(1);
uint8_t cmd0[] = {0x40, 0x00, 0x00, 0x00, 0x00, 0x95}; //Norme de reveil pour les SD, 0x95 est le
    CRC correct pour CMD0
uint8_t resp[7] = {0};
HAL_SPI_Transmit(&hspi3, cmd0, 6, 100);

// Lire 7 octets de reponse
for(int i = 0; i < 7; i++) {
    HAL_SPI_TransmitReceive(&hspi3, &tx, &resp[i], 1, 100);
    myprintf("resp[%d] = %02X\r\n", i, resp[i]);
}
```

---

Une fois la carte réveillée, le microcontrôleur abaisse la ligne CS (GPIO\_PIN\_RESET) pour sélectionner activement la carte microSD sur le bus SPI3. Il lui transmet ensuite la commande CMD0. Cette commande est codée sur 6 octets : 0x40 (l'index de la commande 0), 4 octets d'arguments fixés à 0x00, et 0x95 qui représente une clé de vérification d'intégrité obligatoire (CRC) valide pour cette trame spécifique.

À la suite de cet envoi, le code lit les 7 octets de réponse renvoyés par la carte. Pour valider le bon fonctionnement matériel, la carte microSD doit impérativement retourner la valeur 0x01, signifiant qu'elle est entrée avec succès dans l'état de veille logicielle (In Idle State) et qu'elle accepte désormais le protocole SPI.

Continuons avec le montage du système de fichiers FatFS :

---

```
//Open the file system
fres = f_mount(&FatFs, "", 1); //1=mount now
if (fres != FR_OK) {
    myprintf("f_mount error (%i)\r\n", fres);
    Error_Handler();
}
```

---

L'initialisation bascule vers la couche logicielle supérieure (le middleware FatFS). La fonction `f_mount()` est appelée avec la structure globale `FatFs` afin d'enregistrer et de monter le système de fichiers dans le microcontrôleur.

Le paramètre optionnel 1 force la bibliothèque à exécuter le montage de manière immédiate. FatFS va alors lire physiquement le premier secteur de la carte SD (le Master Boot Record et le Boot Sector) pour vérifier le format de la table d'allocation (FAT16 ou FAT32). Si le résultat (`fres`) est différent de `FR_OK`, le système génère une erreur et bloque l'exécution, évitant l'utilisation d'une carte corrompue ou absente.

Maintenant, on s'assure de la viabilité de l'espace de stockage avant de démarrer la journalisation des mesures du banc avionique :

---

```
//Let's get some statistics from the SD card
DWORD free_clusters, free_sectors, total_sectors;

FATFS* getFreeFs;
fres = f_getfree("", &free_clusters, &getFreeFs);
if (fres != FR_OK) {
    myprintf("f_getfree error (%i)\r\n", fres);
    Error_Handler();
}

//Formula comes from ChaN's documentation
total_sectors = (getFreeFs->n_fatent - 2) * getFreeFs->csize;
free_sectors = free_clusters * getFreeFs->csize;

myprintf("SD card stats:\r\n%10lu KiB total drive space.\r\n%10lu KiB available.\r\n",
        total_sectors / 2, free_sectors / 2);
```

---

La fonction `f_getfree()` interroge le support pour obtenir le nombre de clusters libres (`free_clusters`) restants sur la carte.

Désormais, nous pouvons lire et écrire dans notre carte µSD :

---

```
//Pour ouvrir un fichier en lecture
f_open(&fil, nom_fichier, FA_READ);
//lire les donnees dans le fichier
f_gets((TCHAR*)readBuf, 30, &fil);

//Pour ouvrir dans un fichier en ecriture
f_open(&fil, nom_fichier, FA_WRITE | FA_OPEN_ALWAYS | FA_OPEN_APPEND);
//puis ecrire nos donnees
snprintf(line, sizeof(line), "%.2f;%.2f\r\n", temp, press);
```

---

## 6 UART

### 6.1 Information sur l'UART

Pour la transmission des données de télémétrie et de débogage vers le terminal série du PC, il nous fallait mettre en œuvre la communication UART (**Universal Asynchronous Receiver-Transmitter**). Contrairement à l'I2C et au SPI, l'UART est un protocole de communication série *asynchrone*. La synchronisation est assurée de manière logicielle en configurant une vitesse de transmission identique des deux côtés de la ligne, appelée le **Baud Rate**. Il s'agit également d'un bus **full-duplex** et strictement point à point, reliant directement deux appareils sans besoin d'adressage ni de sélection matérielle.

De par sa nature, la liaison ne requiert que deux lignes physiques :

- **TX (Transmit Data)** : la ligne dédiée à l'émission des données ;
- **RX (Receive Data)** : la ligne dédiée à la réception des données.

Dans le cadre de ce projet, j'ai exploité le périphérique **USART2** de mon STM32L432KC. Les signaux sont routés sur les broches PA-2 (TX) et PA-15 (RX), qui communiquent directement avec le circuit ST-LINK de la carte pour émuler un port COM virtuel sur l'ordinateur via le câble USB. La trame est configurée selon le standard industriel 8N1 : une longueur de mot de 8 bits, aucune parité (*None*) et 1 bit de stop, avec un Baud Rate ajusté à 115 200 baud.

### 6.2 Mise en place de l'UART

D'abord, configurons la communication UART2 dans le fichier `.ioc` en prenant en compte les informations énoncées juste avant :

- le mode : Asynchrone ;
- le Baud Rate : 115200 bits/s (le Terminal Série doit avoir le même) ;
- la longueur du mot : à 8 bits.

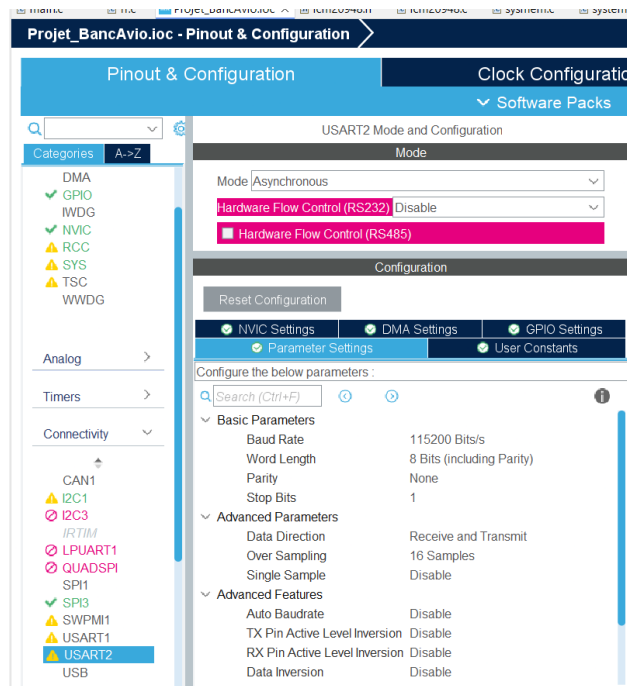


FIGURE 9 – Configuration de l'UART2 dans mon fichier `.ioc`

### 6.3 Au niveau du code

Comment faire afficher un message ? Il nous faut préalablement créer une variable caractère dont on déclare la taille, en fonction du message qu'on souhaite envoyer et qu'on définit juste après, puis nous utilisons la fonction `HAL_UART_Transmit`, tel que :

---

```
char mess[100];
sprintf(mess, "T: %.2f C, P: %.2f hPa \r\n", temp, press);
HAL_UART_Transmit(&huart2, (uint8_t*)mess, strlen(mess), HAL_MAX_DELAY);
```

---

Remarquons que nous devons utiliser trois lignes à chaque fois pour déclarer la taille, le message puis l'ordre alors qu'avec les fichiers ajoutés pour la lecture/écriture sur la microSD, j'ai aperçu un moyen de réduire ces trois lignes en une seule ; en déclarant une fonction `myprintf()` :

---

```
//declaration d'une bibliotheque necessaire
#include <stdarg.h> //for va_list var arg functions
/* ... */
void myprintf(const char *fmt, ...) {
    static char buffer[256];
    va_list args;
    va_start(args, fmt);
    vsnprintf(buffer, sizeof(buffer), fmt, args);
    va_end(args);
    int len = strlen(buffer);
    HAL_UART_Transmit(&huart2, (uint8_t*)buffer, len, -1);
}
```

---

Et voilà, la transmission de message via l'UART est beaucoup plus simple :

---

```
myprintf("Serial Terminal | T: %.2f C, P: %.2f hPa \r\n", temp, press);
```

---

J'en profite pour ajouter que j'ai utilisé le Debug pour tout le projet, et que celui-ci est à mettre en Serial Wire et surtout qu'il se fait par la connexion UART entre le logiciel STM32CubeIDE et notre carte STM32L432KC.

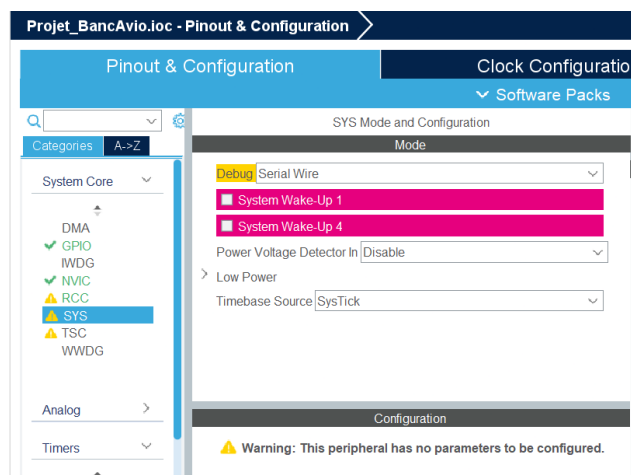


FIGURE 10 – Debug activé, en Serial Wire, pour debugger en connexion UART

## 7 PWM : Modulation de Largeur d'Impulsion

### 7.1 Informations sur le PWM

Pour le pilotage du servomoteur, j'ai mis en œuvre la PWM (**Pulse Width Modulation**, ou Modulation de Largeur d'Impulsion en français).

Le principe du PWM est élémentaire. Sur notre servomoteur, d'après la documentation la période du PWM pour le moteur est de 20 ms, mais sa période d'utilisation se trouve dans l'intervalle [1 ms ; 2 ms] où 1 ms équivaut à la position moteur de -90°, 0° pour 1,5 ms et 90° pour 2 ms. Il est aussi bon de noter que contrairement au reste des composants de mon circuit, il doit opérer en 5V et non en 3,3V.

### 7.2 Mise en place

Dans le cadre de ce projet, j'ai utilisé le **Timer** matériel **TIM16** du STM32L432KC pour générer ce signal de manière totalement autonome (sans bloquer le processeur principal). La fréquence du signal a été fixée à 50 Hz (soit une période de 20 ms). La variation du rapport cyclique permet ainsi de contrôler l'angle du servomoteur sur toute son amplitude, proportionnellement aux calculs de lacet (*yaw*) issus de la centrale inertielle.

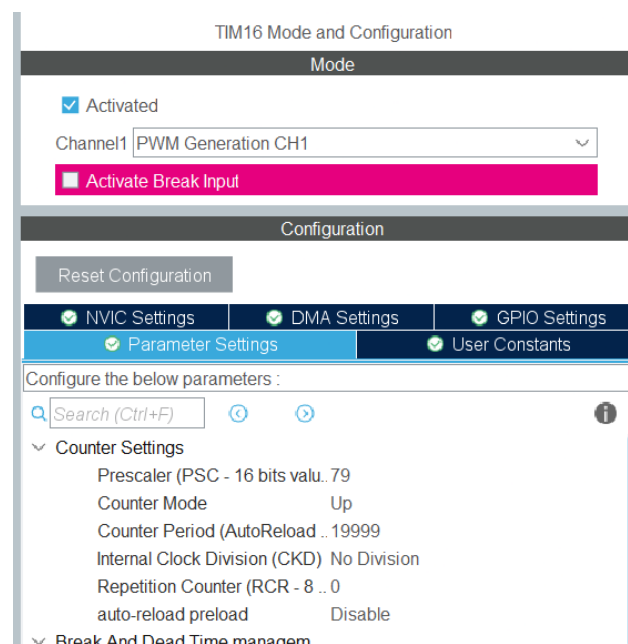


FIGURE 11 – Configuration(1) du PWM sur le fichier *.ioc*



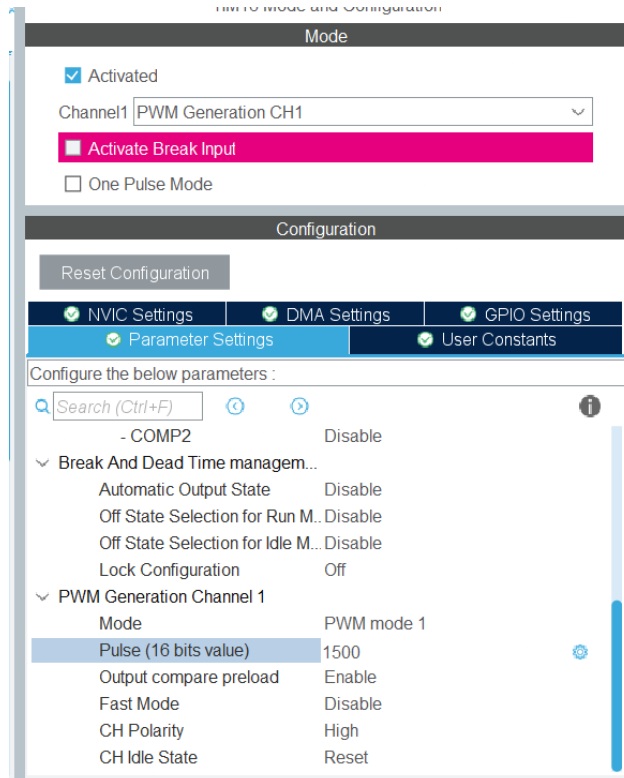


FIGURE 12 – Configuration(2) du PWM sur le fichier .ioc

Comme l'horloge interne de mon processeur est à 80 MHz, je dois ajuster mon **Prescaler** et ma **Counter Period** en fonction.

- le Prescaler : mis à 79, car  $79 + 1$  pour diviser mon horloge interne et avoir une fréquence de timer à 1 MHz, ce qui me laissera une résolution suffisamment grande pour ma commande ;
- la Counter Period à 19 999, pour  $19\,999 + 1$  pour obtenir une période PWM de  $80\text{ MHz} / (79+1) / (19999+1) = 50\text{ Hz}$ .

### 7.3 L'implémentation dans le code

est assez simple. Il ne suffit que de lier l'angle du lacet au rapport cyclique par une fonction :

---

```
uint32_t calcul_rapport_cyclique(float yaw){
    // On sait que le yaw varie entre -180 et +180
    // On veut un rapport cyclique entre 5% de 20 ms = 1 ms (pour -180) -> servoM a -90
    // et 10% de 20 ms = 2 ms (pour +180) -> servoM a +90
    float nb_ticks = 1500 + (yaw / 360) * 1000;
    if (nb_ticks < 1000) nb_ticks = 1000; // Limite inferieure
    if (nb_ticks > 2000) nb_ticks = 2000; // Limite superieure

    return (uint32_t)nb_ticks;
}
```

---

Puis de l'utiliser elle et la fonction `_HAL_TIM_SET_COMPARE`, tel que :

---

```
rapport_cyclique = calcul_rapport_cyclique(yaw);
__HAL_TIM_SET_COMPARE(&htim16, TIM_CHANNEL_1, rapport_cyclique);
```

---

## 8 ARINC

### 8.1 Présentation de l'ARINC429

L'ARINC429 est une norme pour l'aéronautique qui décrit à la fois une architecture, une interface électrique et un protocole pour véhiculer des données numériques (paramètres de vol).

Le message ARINC429 est un bus informatique de 32 bits décomposé de la manière suivante :

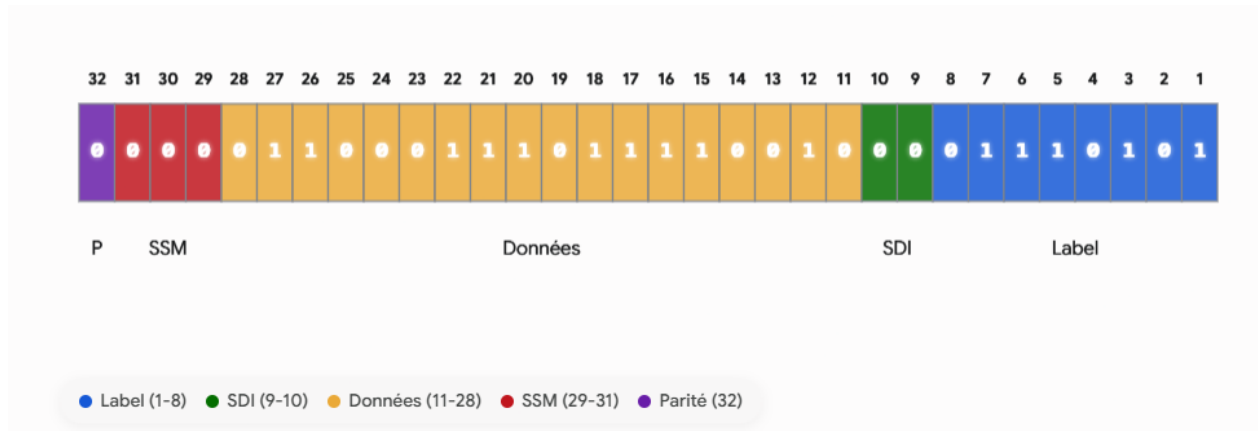


FIGURE 13 – Décomposition de l'ARINC429

- le Label : c'est l'identifiant de la donnée. Il est de 8 bits, qui sont inversés (*1234 -> 4321*) et attention son référencement est en octal. C'est-à-dire que si j'ai le label 256, le chiffre 256 est en octal, en hexadécimal c'est 0xAE et en binaire 0b10101110, ce qui est bien l'inverse de mon exemple de la Figure 11 ;
- le SDI (**S**ource **D**estination **I**dentifier) : il nous est demandé de le garder à 00.
- les Données : c'est là que se trouve l'information pure (que du chiffre) qu'on souhaite communiquer ;
- le SSM (**S**ign/**S**tatus **M**atrix) : il nous est aussi demandé de la garder à 0 ;
- le bit de Parité : il est utilisé pour vérifier que le mot n'a pas été altéré pendant la transmission en assurant l'impairité du nombre de bits à '1' (état haut).

Dans mon projet, la génération du mot ARINC n'est qu'interne au code et on ne vérifie sa valeur que par le Terminal Série ou dans les données envoyées sur ma carte microSD.

### 8.2 Implémentation dans le code

La première difficulté est l'inversion des bits du Label. Pour cela, il est courant d'utiliser une technique où l'on divise les bits du Label en deux parties, puis on les inverse. Après, on redivise en deux ces parties, qu'on réinverse de nouveau, et ainsi de suite. Pour donner une visualisation, prenons 7654 3210 — première division-inversion : 3210 7654 — deuxième : 1032 5476 — Troisième et dernière (car  $2^3 = 8$  bits) : 0123 4567, CQFD.

```
uint8_t inversion_byte(uint8_t byte){  
    //L'ARINC 429 inverse le balel, il donne d'abord le MSB et a la fin le LSB  
    //Si notre label est 429 = 110101101, on doit envoyer 101101011 (0x5B) et pas 110101101 (0x6D)  
    byte = (byte & 0xF0) >> 4 | (byte & 0x0F) << 4;  
    byte = (byte & 0xCC) >> 2 | (byte & 0x33) << 2;  
    byte = (byte & 0xAA) >> 1 | (byte & 0x55) << 1;  
    return byte;  
}
```

La deuxième tâche est de compter le nombre de bits à '1' (état haut) pour s'assurer que notre message soit impair avec le bit de parité :

---

```
int count_set_bits(uint32_t n) {
    int compte = 0;
    while (n) {
        compte += n & 1;
        n >>= 1;
    }
    return compte;
}
```

---

Et finalement, la troisième et dernière difficulté est de rentrer la donnée pure (la valeur de la pression dans notre cas) sans corrompre les autres bits du mot ARINC. Nous utilisons les masques grâce aux marqueurs logiques & et — :

---

```
uint32_t generate_arinc_word(uint32_t pressure, uint8_t label) {
    uint32_t arinc_word = 0; // On part d'un mot vide
    // Le label (Bits 1 a 8)
    uint8_t label_reversed = inversion_byte(label);
    // On implemente ce label dans les 8 premiers bits du mot ARINC
    arinc_word = arinc_word | label_reversed;
    // SDI (Bits 9 et 10)
    // L'enonce demande 0, donc on ne fait rien
    // La donnee pure (Bits 11 a 28)
    uint32_t data_masked = pressure & 0x3FFFF; // Securite pour ne garder que 18 bits de donnees,
        sans debordement
    arinc_word = arinc_word | (data_masked << 10);
    // SSM (Bits 30 et 31)
    // L'enonce demande 0. On ne fait rien.
    // Bit de parite (Bit 32)
    int ones_count = count_set_bits(arinc_word);
    // Si le nombre de '1' est pair, on doit mettre le 32eme bit a '1' pour que le total devienne
    // impair.
    if (ones_count % 2 == 0) {
        arinc_word = arinc_word | ((uint32_t)1 << 31);
        //uint32_t sur le '1' necessaire apparemment pour ne pas avoir un bit signe qui causerait
        // des problemes lors du decalage
    }
    return arinc_word;
}
```

---

Et l'utilisation dans le code :

---

```
uint32_t msg_arinc = 0 ;
uint8_t label_arinc = 0xAE; //256 en octal, 174 en decimal, 0xAE en hexadecimal
/* puis dans le while(1) */
uint32_t press_dec = press * 100; //Pression*100 pour avoir les deux decimales
msg_arinc = generate_arinc_word(press_dec, label_arinc);
```

---

## 9 Mise en place d'une interruption

J'ai mis en place une interruption qui permet de mettre en pause l'acquisition, la remettre en route et même de terminer l'acquisition après un certain nombre d'acquisitions.

### 9.1 Création d'un flag

Pour ce faire, il nous faut juste un bouton-poussoir (BP) branché à une broche mis en GPIO\_EXTI qui détecte le changement de courant et donc que le BP a été pressé. Et on y ajoute un flag d'interruption.

---

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    static uint32_t last_BPclick = 0; //statique pour conserver la valeur entre les appels de la
    fonction, corrigeant ma faute d'anti-rebond
    uint32_t actual_BPclick = 0;
    if(GPIO_Pin == BP_GPIO_EXTI8_Pin)
    {
        actual_BPclick = HAL_GetTick();
        if (actual_BPclick - last_BPclick >= 200)
        {
            stop_logging = !stop_logging; // Inversion de l'etat du flag
            last_BPclick = actual_BPclick; // Mise a jour du temps du dernier clic pour eviter les rebonds
        }
    }
}
```

---

Alors, il suffit d'englober le programme la lecture et affichage des données du while(1) dans une condition d'état (acquisition, ou pause).

### 9.2 Limitation du nombre d'acquisition

Mais je voulais aussi avoir une interruption totale de l'acquisition si nous dépassions un certain nombre d'acquisition, pour ne pas bourrer l'espace de stockage de la carte microSD lorsqu'on oublie le système en marche. Il suffit de définir une valeur max et quand elle est atteinte de faire un `break()`. Nous obtenons ceci :

---

```
while(1)
{
    if (stop_logging == 0){
        /* Programme */
        CTOP++;
        if (CTOP > 900) { //On s'arrete apres 900 mesures, <=> 3 min d'acquisition pour eviter de
            remplir la carte SD
            myprintf(" ~~ limite (900) atteinte, arret de la journalisation ~~ \r\n");
            break;
        }
    }
}
```

---

### 9.3 Ajout d'une LED d'état d'acquisition — Programmation en bare metal

Et pour terminer le tout, j'ai décidé de rajouter une LED pour connaître l'état d'acquisition (en plus du message dans le Terminal Série). La particularité avec cette LED, c'est l'utilisation de la programmation en bare metal, où on ne passe pas par les fonctions HAL mais où nous touchons directement à la couche du CMSIS-Core.

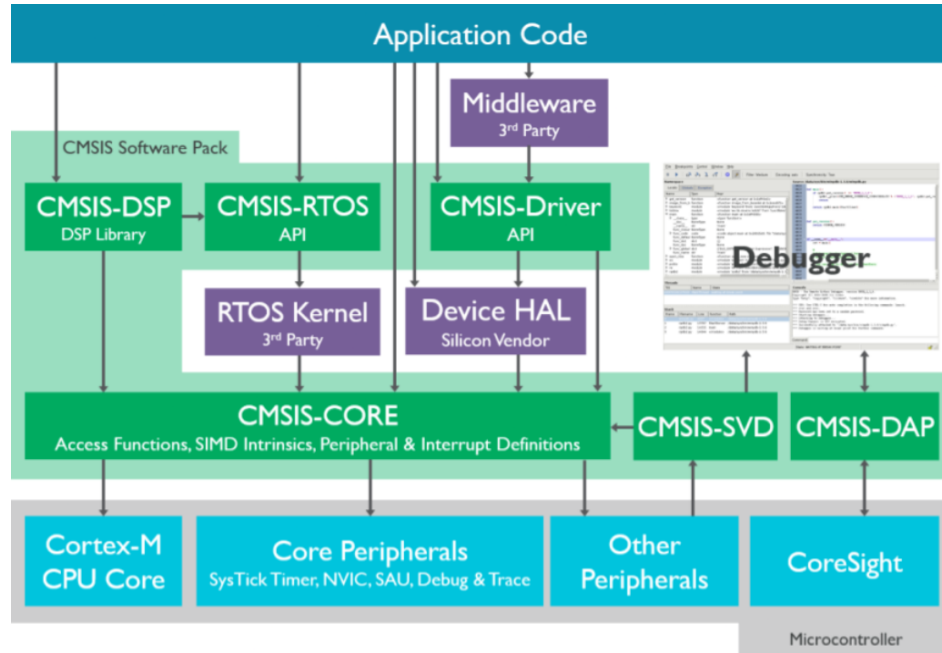


FIGURE 14 – Graphique des couches CMSIS

Pour cela, nous avons besoin du manuel de référence de la STM32L4+. Intéressons-nous aux pages 342 à 345, où nous voyons qu'il nous faut configurer le MODER, l'OTYPER, l'OSPEEDR, le PUPDR et le BSR. Sachant que nous voulons utiliser la broche PA-11 (broche physique D10), il nous faut se référencer à la case n°11 de chacun de ces paramètres :

- MODER : définit la configuration du port. Pour être en mode sortie (**General purpose output mode**) nous devons la mettre à 01 ;
- OTYPER : utile pour activer le push-pull pour stabiliser le signal, ce que nous faisons en mettant OTYPER à 0 ;
- OSPEEDR : le registre de vitesse, nous n'avons pas besoin de haute vitesse, au contraire cela crée des parasites sur le signal, donc on le met à 00 : Low speed ;
- PUPDR : pour l'activation de la résistance de pull-up/down, dont nous n'avons pas besoin car elle serait trop forte pour notre LED qui ne recevrait qu'un maigre courant et à qui j'ai déjà physiquement affecté une résistance de 120 ohm. Donc mis à 00 : No pull-up, pull-down ;
- BSR : elle contient deux registres qui sont à l'état inverse de l'autre. Pour mettre la sortie à l'état haut, je mets le BS11 à '1', et si je souhaite la mettre à l'état bas c'est BR11 qui est mis à '1' ; le '0' n'ayant pas d'action.

---

```

// ~~~ Mode register : MODER ~~~
GPIOA->MODER &= ~(0b11 << 22); // Reset state pour le pin 11; << 22 car pin 11 mais 2 bits par
    pin dans le registre MODER
GPIOA->MODER |= 0b01 << 22; // Mise du pin 11 en mode "output" pour piloter la LED
// ~~~ Output type register : OTYPER ~~~
//Mise du pin 11 en "push-pull" pour un signal stable; <<11 car 1 bit/pin dans l'OTYPER
GPIOA->OTYPER &= ~(0b1 << 11);
// ~~~ Output speed register : OSPEEDR ~~~
// Low speed car c'est une LED, pas besoin de haute frequence, et ca evite les interferences
    electromagnetiques
GPIOA->OSPEEDR &= ~(0b11 << 22);
// ~~~ Pull-up/pull-down register : PUPDR ~~~
// Pas de pull-up ni pull-down pour une sortie LED, une resistance est physiquement presente sur
    mon circuit
GPIOA->PUPDR &= ~(0b11 << 22);
// ~~~ Set/reset register : BSRR ~~~
// Dans le BSRR, il y a le BR pour le reset et le BS pour le set. Un bit = 0 veut dire qu'il ne
    fait rien ("no action")
// Le BR est mis a 1 pour eteindre la LED, le BS n'est pas utilise, il reste a 0 pour ne pas
    allumer la LED
GPIOA->BSRR = (0b1 << 27);

```

---

Puis il nous suffit que de jouer avec le BSRR dans notre boucle while(1). Si l'acquisition est en cours alors la LED est éteinte, si elle est en pause alors la LED est allumée, et si l'acquisition est terminée alors la LED clignote :

---

```

while(1){
    if (stop_logging != prev_stop_logging)
    {
        if (prev_stop_logging == 0) {
            GPIOA->BSRR = (0b1 << 11); //LED allumee en mode pause
            myprintf("Systeme | Acquisition en PAUSE\r\n");
        } else {
            GPIOA->BSRR = (0b1 << 27); //LED eteinte en mode acquisition
            myprintf("Systeme | Acquisition REPRISE\r\n");
        }
        prev_stop_logging = stop_logging;
    }
    /* ... */
}

```

---

À la fin de la boucle while, pour ne pas sortir du main, on remet un autre while(1) secondaire :

---

```

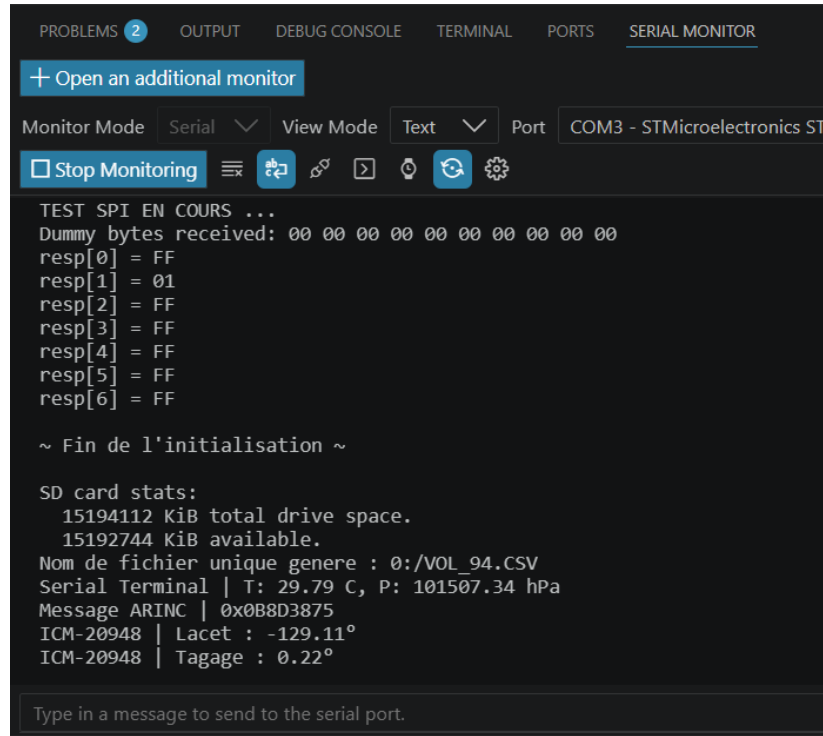
// Bloque le processeur ici indefiniment
while(1) {
    //LED clignotante pour signaler la fin de l'acquisition
    GPIOA->BSRR = (0b1 << 11);
    HAL_Delay(200);
    GPIOA->BSRR = (0b1 << 27);
    HAL_Delay(200);
}

```

---

## 10 Validation et résultats

Le professeur en charge de l'UE a validé les 5 FP du cahier des charges, avec comme seul manquement le mot ARINC qui est certes créé et envoyé sur le terminal série et la carte microSD, mais dont la trame n'est pas générée par le microcontrôleur et ajustée au standard par le multiplexeur pour être lu sur un oscilloscope.



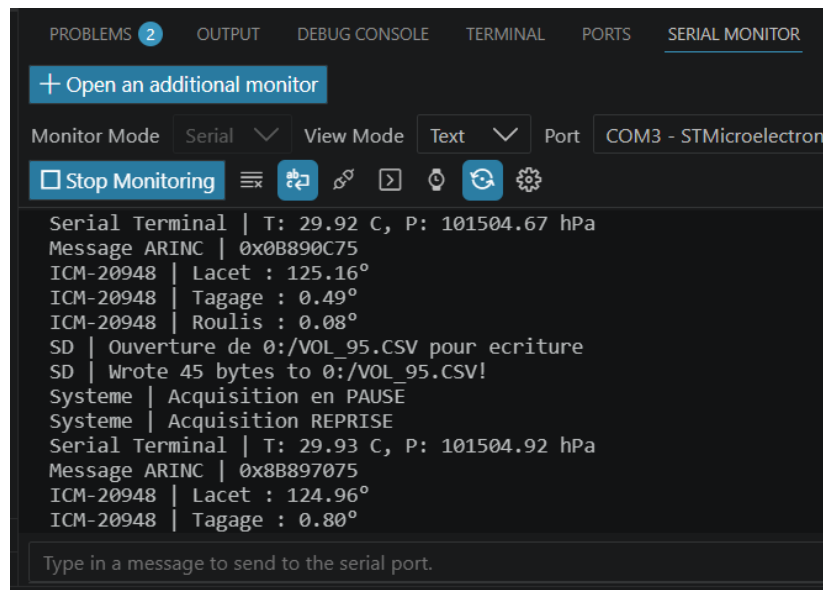
```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR
+ Open an additional monitor
Monitor Mode Serial View Mode Text Port COM3 - STMicroelectronics ST
[Stop Monitoring] [Icons]
TEST SPI EN COURS ...
Dummy bytes received: 00 00 00 00 00 00 00 00 00 00
resp[0] = FF
resp[1] = 01
resp[2] = FF
resp[3] = FF
resp[4] = FF
resp[5] = FF
resp[6] = FF

~ Fin de l'initialisation ~

SD card stats:
  15194112 KiB total drive space.
  15192744 KiB available.
Nom de fichier unique genere : 0:/VOL_94.CSV
Serial Terminal | T: 29.79 C, P: 101507.34 hPa
Message ARINC | 0x0B8D3875
ICM-20948 | Lacet : -129.11°
ICM-20948 | Tagage : 0.22°

Type in a message to send to the serial port.
```

FIGURE 15 – Terminal Série : initialisation de la carte microSD



```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR
+ Open an additional monitor
Monitor Mode Serial View Mode Text Port COM3 - STMicroelectron
[Stop Monitoring] [Icons]
Serial Terminal | T: 29.92 C, P: 101504.67 hPa
Message ARINC | 0x0B890C75
ICM-20948 | Lacet : 125.16°
ICM-20948 | Tagage : 0.49°
ICM-20948 | Roulis : 0.08°
SD | Ouverture de 0:/VOL_95.CSV pour ecriture
SD | Wrote 45 bytes to 0:/VOL_95.CSV!
Systeme | Acquisition en PAUSE
Systeme | Acquisition REPRISE
Serial Terminal | T: 29.93 C, P: 101504.92 hPa
Message ARINC | 0x8B897075
ICM-20948 | Lacet : 124.96°
ICM-20948 | Tagage : 0.80°

Type in a message to send to the serial port.
```

FIGURE 16 – Terminal Série : pause puis reprise de l'acquisition

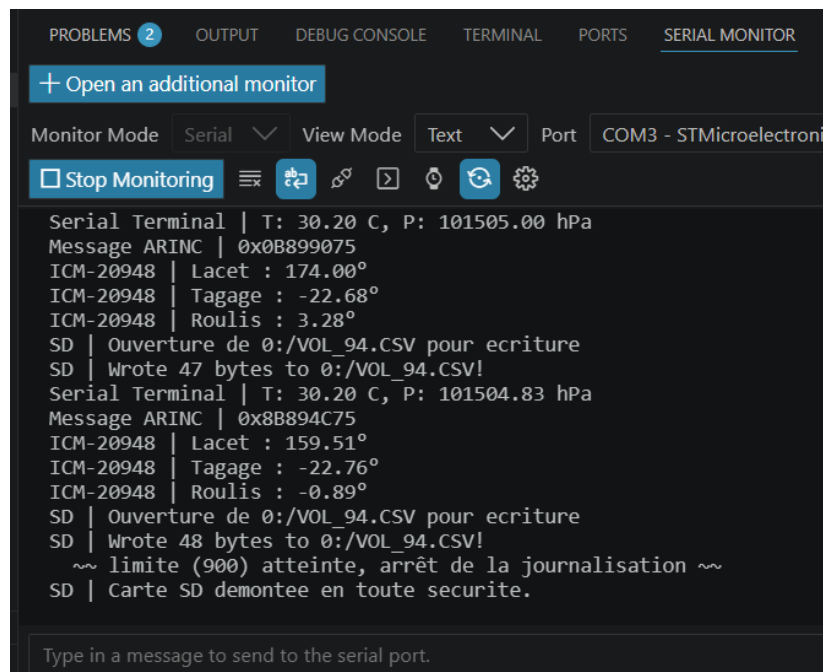


FIGURE 17 – Terminal Série : fin de l’acquisition (*limite atteinte*)

| A1 |       | 29.79     |         |       |       |            |
|----|-------|-----------|---------|-------|-------|------------|
|    | A     | B         | C       | D     | E     | F          |
| 1  | 29.79 | 101507.34 | -129.11 | 0.22  | -0.36 | 0x0B8D3875 |
| 2  | 29.79 | 101506.66 | 155.83  | 0.14  | 0.19  | 0x0B8C2875 |
| 3  | 29.79 | 101507.19 | 155.63  | -0.08 | -0.45 | 0x0B8CFC75 |
| 4  | 29.79 | 101506.67 | 155.69  | 0.06  | -0.36 | 0x8B8C2C75 |
| 5  | 29.79 | 101506.71 | 155.82  | 0.36  | -0.06 | 0x0B8C3C75 |
| 6  | 29.79 | 101506.71 | 156.19  | -0.00 | -0.08 | 0x0B8C3C75 |
| 7  | 29.79 | 101507.21 | 156.39  | -0.47 | -0.61 | 0x0B8D0475 |
| 8  | 29.80 | 101507.57 | 156.52  | -0.14 | -0.42 | 0x0B8D9475 |
| 9  | 29.80 | 101507.22 | 156.58  | 0.06  | -0.31 | 0x0B8D0875 |
| 10 | 29.80 | 101507.40 | 157.81  | 0.03  | 0.14  | 0x8B8D5075 |
| 11 | 29.80 | 101507.40 | 155.12  | -0.17 | 0.28  | 0x8B8D5075 |
| 12 | 29.80 | 101507.92 | 152.49  | 0.17  | -0.56 | 0x0B8E2075 |
| 13 | 29.80 | 101507.73 | 157.59  | -0.00 | 0.14  | 0x8B8DD475 |

FIGURE 18 – Données enregistrées dans la carte microSD

|     |       |           |        |        |       |            |
|-----|-------|-----------|--------|--------|-------|------------|
| 899 | 30.20 | 101504.66 | 170.15 | -19.54 | -7.39 | 0x8B890875 |
| 900 | 30.20 | 101505.00 | 174.00 | -22.68 | 3.28  | 0x0B899075 |
| 901 | 30.20 | 101504.83 | 159.51 | -22.76 | -0.89 | 0x8B894C75 |
| 902 |       |           |        |        |       |            |

< > VOL\_94 +

FIGURE 19 – Limite du nombre d’acquisition



## 11 Bibliographie

Je redonne ici mes sources sur lesquelles s'est basé mon travail :

- *Lien de mon dépôt GitHub* ;
- Lien de la documentation pour le BMP280, liaisons au micro-contrôleur ;
- Lien du dépôt GitHub du BME280, compatible avec le BMP280 ;
- Lien de la documentation pour l'ICM-20948 ;
- Lien du dépôt GitHub pour l'ICM-20948 ;
- Lien de la documentation du SSD1306 ;
- Lien de la documentation de l'écran OLED ;
- Lien du dépôt GitHub pour les fichiers FATFS de la Pmod microSD ;
- d'après la documentation ? ;
- Lien du manuel de référence de la STM32L4+.

Et voici les sources plus générales pour les rappels de cours, les Wikipédias des éléments techniques :

- Lien Wiki sur la programmation bare metal ;
- Lien Wiki pour la communication I2C ;
- Lien Wiki pour la communication SPI ;
- Lien Wiki pour la communication UART ;
- Lien Wiki pour le PWM.

## 12 Annexe — Réponses aux questions

### Q1. Quelle est la fréquence maximale de l'horloge source ?

La fréquence maximale du processeur (ARM Cortex-M4 CPU) est de 80 MHz.

### Q2. Quelle est la taille de la mémoire de code ? celle des variables ?

La mémoire de code correspond à la mémoire Flash : 256 Ko (Kbytes).

La mémoire des variables correspond à la RAM : 64 Ko.

### Q3. Quelles sont les liaisons séries proposées par cette carte ?

Le bloc 'Connectivity' liste les protocoles suivants :

- SPI (x2) et Quad SPI (x1)
- I2C (x2)
- USART (x2) et ULP UART (x1)
- CAN (x1)
- SWP (x1)

### Q4. Que signifie un ADC 16 bits ? + Incohérence

Un ADC (Convertisseur Analogique-Numérique) 16 bits signifie qu'il échantillonne un signal de tension continu sur une résolution de  $2^{16}$  valeurs discrètes (soit 65 536 paliers).

Pour l'incohérence : Le STM32L432 possède matériellement un ADC 12 bits. Cependant, il intègre un module matériel de suréchantillonnage (hardware oversampling) qui combine plusieurs lectures 12 bits consécutives pour en déduire mathématiquement un résultat sur 16 bits.

### Q5. Qu'est-ce qu'un timer ?

Un timer est un périphérique matériel indépendant à l'intérieur du microcontrôleur qui compte les impulsions d'horloge.

Il permet de compter des événements, de créer des délais précis, de déclencher des interruptions ou de générer des signaux carrés (PWM).

### Q6. Quelles pins sont initialisées par défaut ?

- PA2 / PA15 : USART2
- PB 3 : LED verte
- PA13 / PA14 : SWD

### Q7. Quelle est la fréquence de SYSCLK ? Quelle horloge primaire est utilisée ?

Par défaut, le microcontrôleur démarre sur l'horloge interne MSI à 4 MHz.

### Q8. Dans la fonction principale, pourquoi y'a-t-il une boucle infinie ? Que fait-elle ?

Le microcontrôleur ne doit jamais s'arrêter (car il crash sinon) et doit effectuer un programme en boucle ; c'est tout l'intérêt de ce while(1).

### Q9. Quel circuit doit-on réaliser pour permettre à une pin d'allumer une LED lorsqu'un signal GPIO\_PIN\_SET lui est envoyé (comme pour la LED intégrée) ? Dans quel mode doit-on configurer la pin dans l'assistant de configuration ?

Configuration : GPIO Output On doit placer une résistance dans le circuit, soit entre la diode et le pin, soit entre la diode et le GND, car sinon cette dernière serait en surintensité.

**Q10. Même question pour un bouton.**

Pour un bouton, il y a deux configurations, selon qu'on ouvre le pin vers le GND ou vers le + 3.3V. Il faut donc bien configurer avec la résistance de pull-up ou pull-down.

**Q11. Noter les durées importantes permettant le contrôle dans un sens de rotation ou l'autre**

Comme vu dans le rapport :

| Position | Largeur d'impulsion | Rapport cyclique |
|----------|---------------------|------------------|
| -90°     | 1 ms                | 5%               |
| 0°       | 1.5 ms              | 7.5%             |
| 90°      | 2 ms                | 10%              |

TABLE 2 – Correspondance angle / largeur d'impulsion pour le SG90 (issu du rapport)

**Q12. Choisir un timer et effectuer la génération d'un signal PWM pour contrôler le servomoteur. Vous pouvez utiliser le reference manual pour savoir comment paramétrer les registres importants (PSC et ARR).**

Horloge à 80 MHz, Timer 16 bits :

- PSC (Prescaler = 79 → Fréquence timer = 1 MHz (résolution de 1  $\mu$ s)
- ARR (Auto-Reload) = 19 999 → Période = 20 000  $\mu$ s = 20 ms (50 Hz)

**Q13. Etudier les informations importantes de la datasheet du capteur.**

Le LM35 est un capteur de température analogique développé par Texas Instruments. Contrairement aux deux capteurs précédents, il ne dispose pas d'interface numérique intégrée : il délivre directement une tension de sortie proportionnelle à la température mesurée, à raison de 10 mV par degré Celsius.

- Plage de température :  $-55^{\circ}\text{C}$  à  $150^{\circ}\text{C}$
- Sensibilité :  $10\text{mV }^{\circ}\text{C}^{-1}$
- Tension d'alimentation : 4 V à 30 V
- Interface : Analogique

Cette tension est acquise par l'ADC 12 bits intégré au STM32L432KC. La valeur numérique obtenue est ensuite convertie en température via la formule :

$$T(C) = \frac{V_{out} \times 3300}{4095 \times 10} \quad (1)$$